Coinsult

Request your audit at coinsult.net

Advanced Manual
Smart Contract Audit

March 24, 2026

x.com/coinsultaudits

t.me/coinsult_tg

Audit requested by

Zylo Vortex | PowerUp

0x0000000000000000000000000000000000000000

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
Informational	2	0	0	2
Low-Risk	3	1	0	2
Medium-Risk	8	2	0	6
Critical-Risk	8	1	0	7

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
Informational	Does not compromise the functionality of the contract in any way.
Low-Risk	Won't cause any problems, but can be adjusted for improvement.
Medium-Risk	Will likely cause problems and it is recommended to adjust.
Critical-Risk	Will definitely cause problems, this needs to be adjusted.

Audit Summary

Project	Zylo Vortex | PowerUp
Website	http://www.zylovortex.live/
Blockchain	BSC (BEP-20)
Source Code	https://bscscan.com/address/0x0000000000000000000000000000000
Contract Address	0x0000000000000000000000000000000000000

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code. Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

Slither: Solidity static analysis frameworkRemix: IDE Developer ToolCWE: Common Weakness EnumerationSWC: Smart Contract Weakness Classification and Test CasesDEX: Testnet Blockchains

Table of Contents

Global Overview 2

Audit Summary 3

Audit Scope 4

Detailed Findings

Broken random ID generation (DoS) 6

Pseudo-randomness is manipulable 7

Owner can send tokens via emergencyWithdrawToken 8

Owner can change core time parameters 9

Owner can arbitrarily set reward rates 10

External milestone contract can DoS 11

Unsafe ERC20 interactions (no SafeERC20) 12

tx.origin blocks smart wallets 13

Whitelist deposit uses tx.origin + onlyOwner 14

Claim event emits wrong values/order 15

Potential underflow in totalActivePowerUpToken 16

Division by zero risk in updatePowerUp 17

getPerformanceMultiplier logic inconsistent 18

Unbounded loops can cause gas DoS 19

Milestone updates inside loops (gas/DoS) 20

Whitelist array index getter can revert 21

Whitelist count can underflow on removal 22

No validation for token/milestone addresses 23

Misleading/incorrect naming and counters 24

Test-time constants in production logic 25

No events for critical admin changes 26

Disclaimer 27

Broken random ID generation (DoS)

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-001: Resolved	getSixDigitRandomUserId() never sets idExists[randomID]=true, so duplicates are possible. Also it uses recursion in a while loop; if collisions happen it can recurse until out-of-gas and block registrations. Duplicate IDs overwrite getZyloIDtoaddress mapping, breaking identity mapping.

Code Snippet

```
while (idExists[randomID]) {
  randomID = getSixDigitRandomUserId();
}
return randomID; // idExists never set
```

Recommendation

Mark IDs as used: set idExists[randomID]=true before returning. Avoid recursion; use an iterative loop with a max-tries limit. Consider using a monotonically increasing counter or off-chain assignment for IDs.

Note

Random/recursive 6-digit ID generation removed. IDs are now assigned via monotonically increasing nextUserId in inceptNow/constructor, eliminating collisions/DoS and mapping overwrite risk.

Pseudo-randomness is manipulable

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-002: Resolved	Randomness uses block.timestamp/prevrandao/difficulty and msg.sender, which miners/validators and users can influence. This affects assetsNo and dailyPowerUp percentage, enabling manipulation of rewards/parameters and unfair outcomes.

Code Snippet

```
keccak256(abi.encodePacked(block.timestamp, block.prevrandao, msg.sender))
```

Recommendation

Use a verifiable randomness source (e.g., Chainlink VRF) or commit-reveal scheme. Do not use block variables for value-bearing randomness.

Note

No block-variable-based randomness remains (no keccak256(block.timestamp/prevrandao/msg.sender) usage). Asset selection now uses deterministic getNextAsset().

Owner can send tokens via emergencyWithdrawToken

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-003: Resolved	Owner can transfer any ERC20 held by this contract to themselves. If tokens are ever sent here (accidentally or via future changes), funds can be drained. This is a strong centralization/rug risk.

Code Snippet

```
function emergencyWithdrawToken(address _token, uint256 _amount) external onlyOwner {
  IERC20(_token).transfer(owner(), _amount);
}
```

Recommendation

Add timelock + multisig and emit events. Consider a withdrawal allowlist.

Note

emergencyWithdrawToken function is no longer present in the new code, removing the owner token-drain vector described.

Owner can change core time parameters

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-004: Resolved	Owner can arbitrarily change inceptStartTime and dayReqOnStart, which changes month/day calculations and reward/claim behavior. This can retroactively alter user outcomes and enable unfair reward manipulation.

Code Snippet

```
function updateTime(uint256 _inceptStartTime) external onlyOwner{
  inceptStartTime = _inceptStartTime;
}
```

Recommendation

Make time parameters immutable after launch, or only allow one-time initialization. If changes are needed, use timelock governance and clear public disclosure.

Note

inceptStartTime is now immutable and set in the constructor. updateTime is removed, so owner can't arbitrarily change the start time.

Owner can arbitrarily set reward rates

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-005: Resolved	Owner can set dailyPowerUpAmount for any month and adjust percentages/requirements. This can drastically change payouts, lock/unlock levels, and alter economics after users deposit.

Code Snippet

```
function updateDailyPowerUpAmount(uint256 _month, uint256 _amount) external onlyOwner {
  dailyPowerUpAmount[_month] = _amount;
}
```

Recommendation

Hardcode or cap parameters, or restrict changes to a timelocked multisig. Add upper bounds (e.g., max % per day/month) and emit events for all parameter changes.

Note

> Owner setters for reward rates/percentages (e.g., updateDailyPowerUpAmount) are not present. dailyPowerUpAmount is computed internally by updatePowerUp() from powerUpPattern and totals.

External milestone contract can DoS

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-006: Resolved	Calls to ZyloPowerUpMilestone are unchecked and can revert, blocking powerUp/claim flows. If milestone address is unset or malicious, users may be unable to interact.

Code Snippet

```
ZyloPowerUpMilestone.updateMilestoneReq(_user);
ZyloPowerUpMilestone.updateTopPowerUP(...);
```

Recommendation

Require milestone address to be set before use, or wrap calls in try/catch to prevent full DoS. Consider making milestone updates optional or callable by a separate process.

Note

Milestone calls are now guarded: _safeMilestoneUpdate() and updateUnitTeam() wrap external calls in try/catch and also no-op when milestone is zero, preventing full-flow DoS on revert/unset.

Unsafe ERC20 interactions (no SafeERC20)

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-007: Pending	Contract assumes ERC20 transfer/transferFrom returns bool. Many tokens (e.g., USDT) don't return bool and will revert or behave unexpectedly. emergencyWithdrawToken also ignores return value.

Code Snippet

```
bool success = token.transferFrom(_user, address(ZyloPowerUpMilestone), _amount);
require(success, "Transfer failed");
```

Recommendation

Use OpenZeppelin SafeERC20 (safeTransfer/safeTransferFrom). Always handle non-standard ERC20s safely and check return values consistently.

Note

SafeERC20 is used for safeTransferFrom, but _forwardToMilestone uses raw token.transfer(...) in a try/catch. This still breaks for non-standard ERC20s that don't return bool (e.g., USDT), and may silently fail.

tx.origin blocks smart wallets

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-008: Resolved	Using require(msg.sender==tx.origin) blocks multisigs, smart wallets, account abstraction, and relayers. It also provides weak security and can break integrations and user safety tooling.

Code Snippet

```
require(msg.sender == tx.origin, "External Access Not Allowed");
```

Recommendation

Remove tx.origin checks. If bot protection is needed, use allowlists, EIP-712 signatures, or rate limits. Support smart wallets for safer custody.

Note

No tx.origin checks exist in the new code; smart wallets/multisigs are no longer blocked by that pattern.

Whitelist deposit uses tx.origin + onlyOwner

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-009: Resolved	whitelistPowerUp is onlyOwner but also requires msg.sender==tx.origin, preventing execution via multisig (recommended for admin). This increases operational risk and centralization.

Code Snippet

```
function whitelistPowerUp(...) external onlyOwner {
  require(msg.sender == tx.origin, ...);
}
```

Recommendation

Remove tx.origin restriction for admin functions. Use a multisig owner and optionally a timelock for sensitive actions.

Note

> whitelistPowerUp is onlyOwner and no longer requires msg.sender == tx.origin, enabling multisig/time-lock owners.

Claim event emits wrong values/order

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-010: Resolved	ClaimSelfPowerUnits event is defined as (user, amount, unit) but emitted as (user, unit, remainingCap). Off-chain accounting/UIs will misread claims, harming transparency and user trust.

Code Snippet

```
event ClaimSelfPowerUnits(address indexed _user, uint256 amount, uint256 unit);
...
emit ClaimSelfPowerUnits(_user, _unit, remainingCap);
```

Recommendation

Emit correct parameters: emit ClaimSelfPowerUnits(_user, claimAmount, _unit). Consider adding a new event for remainingCap if needed.

Note

The prior ClaimSelfPowerUnits event mismatch is no longer applicable; that event/emission pattern is removed. Claim flow now emits RemainingUnitCap(_user,_unit,claimAmount).

Potential underflow in totalActivePowerUpToken

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-011: Pending	totalActivePowerUpToken is decreased in multiple places and can be decremented again when unPowerUp triggers. If accounting gets out of sync, Solidity 0.8 will revert, causing claim DoS.

Code Snippet

```
totalActivePowerUpToken -= burnToken;
...
if (alreadyClaimed + claimAmount >= maxCap) {
  totalActivePowerUpToken -= c.powerUpActiveToken;
}
```

Recommendation

Add invariant checks and use safe min() logic before subtracting. Consider tracking per-position active totals and updating once. Add tests for edge cases where burnToken exceeds active token.

Note

totalActivePowerUpToken is decremented in both _processSelfClaim (burnToken) and again on completion in _processReferralAndCompletion (remainingActive). While burnToken is capped to oldActive, inconsistent accounting can still revert (Solidity 0.8) if totalActivePowerUpToken is already lower than the subtraction amount.

Division by zero risk in updatePowerUp

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-012: Resolved	updatePowerUp divides by totalActivePowerUpToken twice. If totalActivePowerUpToken becomes 0 (e.g., all positions completed), calling updatePowerUp will revert and can block flows that call it.

Code Snippet

```
dailyPowerUpAmount[_month] = ... / totalActivePowerUpToken);
```

Recommendation

Guard: require(totalActivePowerUpToken>0) before computing. If zero, set dailyPowerUpAmount to 0 and skip updates.

Note

updatePowerUp now guards totalActivePowerUpToken==0 and sets dailyPowerUpAmount[_month]=0, avoiding division-by-zero.

getPerformanceMultiplier logic inconsistent

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-013: Resolved	Function requires month > 1 but then checks month>=1, month>=2, month>=3. This makes “Month 1” branch unreachable and likely breaks intended vesting/claim rules for whitelisted users.

Code Snippet

```
require(month > 1, "Sorry, you can not claim");
if (month >= 1) { ... }
```

Recommendation

Fix the condition to match intended schedule (e.g., require(month >= 1) or use explicit month==1/2/3). Add unit tests for each month boundary.

Note

getPerformanceMultiplier now uses require(month > 0) and then month>=1/2/3 branches, making Month 1 reachable and removing the contradictory require(month>1) logic.

Unbounded loops can cause gas DoS

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-014: Pending	Several functions iterate over dynamic arrays/mappings (powerUpDetails length, down-line users, nested loops). As data grows, view and state-changing calls may exceed gas limits, preventing claims or calculations.

Code Snippet

```
for (uint256 i = 0; i < length; i++) { ... }
for (uint256 i = 0; i < totalUsers; i++) { ... for (uint256 index = 0; index < powerUpLen;
index++) ... }
```

Recommendation

Add pagination for getters and avoid heavy on-chain aggregation. For state changes, limit per-tx work or use checkpointing. Move large computations off-chain.

Note

Unbounded loops remain in multiple paths (e.g., getTotalSelfPowerUp/currentSelfPowerUp iterate over userPowerUpDetails length; hasActivePowerUp scans all entries; updateInviterChain/updateUnit-Team/updateInviterPowerUp loop up to chainLength). One view function was paginated, but core loops can still grow and cause gas issues.

Milestone updates inside loops (gas/DoS)

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-015: Resolved	updateUnitTeam calls ZyloPowerUpMilestone.updateMilestoneReq inside a loop (up to 3 times). If milestone is expensive or reverts, deposits/claims can fail due to gas or revert.

Code Snippet

```
if(level < 3){
  ZyloPowerUpMilestone.updateMilestoneReq(_inviter);
}
```

Recommendation

Batch milestone updates or emit events and let milestone contract process asynchronously. Use try/catch to avoid blocking core flows.

Note

Milestone updates inside updateUnitTeam are now wrapped in try/catch, so milestone reverts won't block deposits/updates (though gas cost still exists).

Whitelist array index getter can revert

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-016: Resolved	getWhitelistAddress does not bounds-check index. Calls with an invalid index revert, which can break frontends and integrations relying on safe reads.

Code Snippet

```
function getWhitelistAddress(uint256 unit, uint256 index) external view returns (address) {
  return whitelistArray[unit][index];
}
```

Recommendation

Add require(index < whitelistArray[unit].length). Alternatively return (address(0), false) style safe getter.

Note

A safe getter getWhitelistAddressSafe(unit,index) was added that returns (address(0),false) instead of reverting on out-of-bounds.

Whitelist count can underflow on removal

Low-Risk *- Won't cause any problems, but can be adjusted for improvement.*

Error Code	Description
AUD-017: Resolved	removeFromWhitelist decrements whitelistCount[unit] without extra safety. If state becomes inconsistent (e.g., manual storage corruption or future code changes), it will revert and lock admin cleanup.

Code Snippet

```
whitelistCount[unit]--;
```

Recommendation

Use checked logic: require(whitelistCount[unit]>0) before decrement. Keep whitelistCount derived from array length to avoid mismatch.

Note

removeFromWhitelist now checks whitelistCount[unit] > 0 before decrementing, preventing underflow revert.

No validation for token/milestone addresses

Low-Risk *- Won't cause any problems, but can be adjusted for improvement.*

Error Code	Description
AUD-018: Resolved	Constructor and setMilestone do not validate non-zero addresses or contract code. Setting wrong addresses can permanently break transfers or milestone calls.

Code Snippet

```
constructor(address _token, address _inceptRootAddress) { token = IERC20(_token); ... }
function setMilestone(address _ZyloPowerUpMilestone) external onlyOwner { ... }
```

Recommendation

Require non-zero and optionally check code length (address.code.length>0). Emit events when changing critical addresses.

Note

Constructor validates token/root non-zero and token.code.length>0. setMilestone now requires non-zero and code length > 0, reducing misconfiguration risk.

Misleading/incorrect naming and counters

Low-Risk *- Won't cause any problems, but can be adjusted for improvement.*

Error Code	Description
AUD-019: Pending	totalPowerUpUsers increments on every powerUp, not unique users. totalInspectNowUsers name suggests “incept” but increments in inceptNow. Misleading metrics can confuse users and auditors.

Code Snippet

```
totalPowerUpUsers += 1; // on each deposit
```

Recommendation

Track unique users separately (e.g., mapping seen). Rename counters to reflect meaning (e.g., totalPowerUpTx).

Note

Counters/naming issues persist: totalPowerUpUsers increments only when powerUPTotalToken==0 (unique-ish), but totalInspectNowUsers naming still suggests 'inspect' while used for inceptNow registrations; other metrics may still be misleading.

Test-time constants in production logic

Informational *- Does not compromise the functionality of the contract in any way.*

Error Code	Description
AUD-020: Resolved	secondsInOneDay/Week/Month are set to 120/840/3600 (test values). If deployed to production, reward timing and vesting will be drastically different than expected.

Code Snippet

```
uint256 public secondsInOneDay = 120;
uint256 public secondsInOneMonth = 3600;
```

Recommendation

Use real time constants (1 days/1 weeks/30 days) for production or make them immutable and clearly documented. If configurable, restrict changes via timelock.

Note

Time constants are now set to production-like values: secondsInOneDay=1 days, secondsInOne-Week=1 weeks, secondsInOneMonth=30 days (instead of test values).

No events for critical admin changes

Informational *- Does not compromise the functionality of the contract in any way.*

Error Code	Description
AUD-021: Resolved	Admin functions change key parameters (percentages, requirements, time, milestone address) without emitting events, reducing transparency and making it hard for users to detect changes.

Code Snippet

```
function updateAirDropPercentage(uint256 index, uint256 value) external onlyOwner { airDrop-
Percentage[index] = value; }
```

Recommendation

Emit events for every admin parameter change (oldValue,newValue). Consider timelock + public announcement policy.

Note

Critical admin change event added for milestone updates (MilestoneUpdated). Many prior admin setters are removed; remaining critical change (setMilestone) now emits an event.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.